

ISE Report: Config Tuning a Retrieval Component in RAG Pipeline of Legal Domain

1. Introduction

Legal professionals routinely work with large volumes of complex documents, including contracts, statutes, and case materials. A significant portion of this work involves locating relevant clauses, interpreting legal language, and cross-referencing provisions across documents, which is both time-consuming and error-prone.

Traditional keyword-based search tools provide limited support in this context. Legal queries often involve nuanced phrasing, implicit references, or structural elements such as sections and clauses that are not easily captured by simple lexical matching. As a result, those tasked with document review must frequently perform manual review, navigating long documents to identify the exact span of relevant information.

Recent advances in Retrieval-Augmented Generation (RAG) have enabled effective document-level question answering. However, the performance of RAG systems is highly sensitive to their internal configurations, such as text chunking strategies, embedding selections, and retrieval thresholds. Currently, most RAG research and system optimisations are tailored for cloud-based environments, leveraging the vast computational power and high-performance proprietary models available online.

To address these challenges, we develop PowerLLM, a local RAG-based system designed for legal document analysis. This application is a lightweight local RAG system built on Qwen-3.5-4B, with retrieval and generation pipeline. PowerLLM aims to address secure and efficient processing of legal texts using lightweight language models that can operate under constrained computational environments. However, such settings introduce additional challenges: smaller models have limited context windows and reduced tolerance for noisy or incomplete retrieval results, making system performance more dependent on effective configuration choices.

Specifically, the effectiveness of PowerLLM is strongly influenced by configuration decisions such as chunking strategies (e.g., chunk size and overlap) and retrieval parameters (e.g., the number of retrieved passages). While prior studies have shown that these factors significantly impact RAG performance, existing findings are primarily based on general-domain datasets, and their applicability to legal document processing remains unclear.

To this end, we investigate configuration tuning for the retrieval component of RAG systems in legal document processing. We systematically evaluate key parameters, including chunking strategies and retrieval settings, and analyse their impact on retrieval quality. Based on these findings, we explore practical tuning strategies to improve retrieval effectiveness under domain-specific and resource-constrained conditions.

2. Related Work

Existing work on improving RAG systems increasingly focuses on tuning system-level configurations rather than modifying the underlying language model. These configurations include chunk size, chunk overlap, retrieval depth, embedding model choice, reranking strategy, and pipeline structure. Since RAG performance depends on how documents are segmented, retrieved, and passed to the generator, configuration choices can significantly affect both retrieval quality and answer accuracy.

One line of work studies automated RAG pipeline optimisation. AutoRAG automatically evaluates different combinations of RAG modules and selects a suitable pipeline for a given dataset [1]. Similarly,

AutoRAG-HP focuses on automatic hyperparameter tuning for RAG systems, reducing the need for manual trial and error [2]. These approaches are effective in exploring a broader configuration space more systematically than manual tuning. However, they typically require prepared evaluation data and repeated experimental runs, which can be costly for lightweight local systems.

Another line of work focuses on evaluation-driven RAG improvement. RAGAS provides reference-free metrics for evaluating RAG pipelines, while ARES evaluates systems along dimensions such as context relevance, answer faithfulness, and answer relevance [3], [4]. These frameworks facilitate systematic comparison of different RAG configurations. A key limitation is their reliance on LLM-as-a-judge evaluation. While this reduces the need for human annotation, it can become computationally expensive when applied to large datasets across multiple rounds of configuration tuning. Each configuration may require multiple LLM calls to assess retrieved contexts and generated answers, resulting in high token consumption and long evaluation time. This makes them less suitable for rapid iteration in resource-constrained local RAG experiments.

Document chunking is also an important tuning factor. Prior studies show that chunking strategy can significantly affect retrieval performance: overly small chunks may lose contextual information, while overly large chunks may introduce irrelevant content [5], [6]. This is relevant for legal document question answering, where answers often depend on surrounding clauses and precise wording.

Instead of relying on automated optimisation frameworks, we adopt a manual empirical tuning approach. We focus on a small set of key RAG configuration parameters, including chunk size, chunk overlap, and retrieval depth, and evaluate their impact through controlled experiments using retrieval and coverage-based metrics.

3. Solution

3.1 Base Legal RAG Pipeline

The retrieval system in the base legal RAG pipeline adopts a hybrid architecture that combines dense and sparse retrieval methods to exploit both semantic and lexical matching signals. In our design, a dense retriever is used to capture semantic similarity between queries and document chunks, while a BM25-based sparse retriever is employed to preserve exact term matching and handle domain-specific terminology common in legal text.

The results from both retrieval components are aggregated using the Reciprocal Rank Fusion (RRF) algorithm, which merges ranked lists in a score-agnostic manner.

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

Where R denotes the set of retrieval result lists, $\text{rank}_r(d)$ is the rank position of chunk d in retriever r , and k is a smoothing constant that reduces the dominance of top-ranked items while still rewarding consistently well-ranked candidates. In the retrieval setting, R here contains two ranked lists: one from dense vector similarity retrieval and one from BM25 retrieval.

The final retrieval quality is also influenced by the ratio between the BM25 top- k candidates and the dense similarity top- k candidates. This ratio controls the relative contribution of lexical and semantic retrieval signals in the fused ranking, and therefore needs to be tuned based on the downstream retrieval objective.

3.2 Budget-Constrained Tuning Strategy

Our preliminary experiments revealed a mechanical correlation between chunk size and quality scores, leading to a 'chunk size bias' during configuration tuning. This phenomenon tends to skew the optimisation process toward larger chunk sizes, which may reflect an artefact of the evaluation metric rather than a genuine improvement in performance.

To ensure fair comparison across configurations, we adopt a budget-constrained tuning strategy based on latency.

We first define a maximum acceptable latency T_{max} , and measure the average end-to-end processing time per query under different configurations:

$$t(c) = \mathbb{E}[\text{latency per query under configuration } c]$$

We then enforce the latency constraint (prune if not meet):

$$t(c) \leq T_{max}$$

In practice, exact token counts are not directly accessible in our pipeline. We therefore approximate the input size using the total number of characters in the retrieved context:

$$\text{context}_{\text{char}} \leq B_{\text{char}}$$

Based on empirical measurements under $T_{max} = 24s$ we set:

$$B_{\text{char}} \approx 22,000$$

This corresponds to roughly 5,500 tokens using a standard character-to-token approximation.

Hyperparameter tuning is then performed under this fixed budget constraint:

$$\max_{\theta} \text{quality_score}(\theta)$$

subject to:

$$\text{context}_{\text{char}}(\theta) \leq B_{\text{char}}$$

This ensures that all configurations are evaluated under the same inference cost, preventing larger chunk sizes from gaining an artificial advantage through increased context volume.

Within this constrained search space, we tune the retrieval and chunking-related hyperparameters using multiple search strategies, including Grid Search, Random Search, and Tree-structured Parzen Estimator (TPE). For each candidate configuration, the retrieved context size is first checked against the fixed character budget. Configurations that exceed the budget are pruned before evaluation, while the remaining configurations are evaluated using the composite quality score.

Grid Search is used as a simple baseline to examine the behaviour of predefined hyperparameter combinations. Random Search provides broader coverage of the discrete search space under the same evaluation budget, while TPE is included as a model-based optimisation method that adaptively samples configurations based on previous trials. Comparing these strategies allows us to assess whether more adaptive optimisation provides a practical advantage in this setting, or whether broader random exploration is more robust given the discreteness of the search space and the noise in the evaluation metric.

4. Setup

4.1 Datasets

Since the main component of PowerLLM is a retrieval module designed for the legal domain, we require datasets that are suitable for evaluating retrieval performance on legal documents.

For configuration tuning, we use the datasets provided by LegalBench-RAG [7], which consist of four legal-domain datasets. We select these datasets because they cover diverse types of legal documents, including non-disclosure agreements, private contracts, mergers and acquisitions documents, and privacy policies. Such diversity allows us to evaluate retrieval performance across different legal contexts and document structures.

LegalBench-RAG restructures the original LegalBench data into query-document pairs. This format improves the evaluation process for retrieval-based tasks.

Table 1. Summary of Datasets

Q&A Count	Dataset Name	Dataset Description
946	ContractNLI	NDA related documents
4042	Contract Understanding Atticus Dataset (CUAD)	Private Contracts
1676	M&A Understanding Dataset	M&A documents of public companies
194	Privacy QA	Privacy policies of consumer apps
6858	4 datasets	Variety of legal documents

Due to computational constraints, the full tuning and validation datasets were sampled to 200 questions each. The mixed benchmark contains 70 questions from CUAD, 50 from MAUD, 50 from ContractNLI, and 30 from PrivacyQA. CUAD and MAUD are intentionally overrepresented, as they pose greater challenges for the RAG pipeline.

For hyperparameter tuning, each trial was evaluated on a fixed 50-question subset of the 200-question tuning split. This subset was constructed by selecting 10 contexts and up to 5 questions per context to preserve cache reuse during chunking and retrieval experiments.

The trial result is later verified with validation dataset (200 questions).

4.2 Optimisation Settings

This system utilises Optuna with the Tree-structured Parzen Estimator (TPE) as the primary search method. However, we observed that under our setting, TPE tends to prematurely concentrate on a narrow region of the search space. We suspect that it might be due to the discrete nature of the hyperparameters and the noise in the evaluation metric.

To mitigate this issue and improve exploration, we additionally experimented with Random Search and Grid Search. The python script we provided support Random Search, Grid Search and TPE.

4.3 Metrics

The evaluation considers four aspects: retrieval effectiveness, ranking quality, evidence coverage, and efficiency.

- **Recall@k** (**Recall@10**, **Recall@20**, **Recall@final_k**): Proportion of gold answer spans covered within the top-k retrieved results.
- **Hit@k** (**Hit@1**, **Hit@3**, **Hit@5**): Whether at least one gold answer span is covered within the top-k retrieved results.
- **Precision@k** (**Precision@5**, **Precision@10**, **Precision@20**, **Precision@final_k**): Proportion of retrieved chunks that fully cover at least one gold answer span.
- **F1@k** (**F1@10**, **F1@final_k**): Harmonic mean of Recall@k and Precision@k.
- **MRR** (**Mean Reciprocal Rank**): Measures how early the first relevant chunk appears in the ranked retrieval list.
- **Fully Covered** / **Partial+** / **Not Covered**: Evidence coverage metrics indicating whether all, at least one, or none of the gold answer spans are covered by the final retrieved context.
- **Overlap** (**Mean_overlap_ratio_at_final_k_mean**): Average maximum character-level overlap ratio between final retrieved chunks and gold answer spans.
- **Latency**: Average retrieval time per question.

4.4 Quality Score

The quality score is designed to aggregate multiple aspects of retrieval performance into a single optimisation objective. The weighting scheme reflects the relative importance of different evaluation criteria in legal document retrieval, with a primary emphasis on evidence coverage and recall, while also considering ranking quality and retrieval efficiency.

This design is motivated by the characteristics of legal applications, where failing to retrieve relevant evidence can have more severe consequences than retrieving additional irrelevant context. As a result, recall- and coverage-oriented metrics are assigned higher weights to prioritise comprehensive evidence retrieval.

Ranking-based metrics are retained with moderate weights to reward configurations that place relevant evidence earlier in the retrieved list. The overlap metric is included as a lightweight auxiliary signal for partial evidence matching, but is assigned a smaller weight as it is less directly interpretable than item-level coverage metrics.

$$\begin{aligned} \text{quality_score} = & 0.30 \times \text{recall_at_final_k} \\ & + 0.18 \times \text{partial_or_better_rate} \\ & + 0.22 \times \text{fully_covered_at_final_k} \\ & + 0.15 \times \text{mrr_mean} \\ & + 0.10 \times \text{hit_at_5} \\ & + 0.05 \times \text{mean_overlap_ratio_at_final_k} \end{aligned}$$

5. Experiments

5.1 Experimental Design

We compare two tuning settings: an unconstrained empirical grid search baseline and budget-constrained optimisation using Random Search and TPE. The unconstrained setting evaluates retrieval quality without limiting the retrieved context size, while the budget-constrained setting prunes configurations whose estimated context size exceeds 22,000 characters. This design allows us to distinguish genuine retrieval improvement from gains caused by increased context volume.

5.2 Results

Table 2. Settings

Method	Search strategy	Budget constraint	Purpose
Empirical Grid Search baseline	Fixed manual grid	No	Represents conventional manual tuning
Proposed approach	Random Search / TPE under budget	Yes	Controls inference cost and context volume

Table 3. Baseline(Grid Search) top-5 Without Budget Limit

Rank	Trial	Score	Chunk Size	Similarity k	BM25 k	Final k	Estimated Context Chars
1	63	0.5610	2000	12	40	14	28,000
2	6	0.5601	2000	12	80	14	28,000
3	41	0.5594	2000	12	60	14	28,000
4	57	0.5395	2000	8	40	14	28,000
5	97	0.5379	2000	8	60	14	28,000

The no-budget empirical grid baseline achieved the highest retrieval score, but its best configurations consistently selected the largest chunk size and final retrieval window available in the grid. This indicates that unconstrained manual tuning tends to improve recall by expanding the context volume, often exceeding the intended inference budget. In contrast, the proposed budget-aware search explicitly controls the context size, trading a small amount of retrieval quality for bounded inference cost and predictable context usage.

The highest score within token budget limit, drops to 0.5026.

Rank	Trial	Score	Chunk Size	Similarity k	BM25 k	Final k	Estimated Context Chars
18	34	0.5026	2000	12	80	10	20,000

Table 4. Benchmark of the Highest Score Config within Token Budget Limit

Metrics	CUAD	PrivacyQA	ContractNLI	MAUD	Mixed Validation
Recall@10	0.6343	0.6855	0.9800	0.2880	0.6365
Hit@1	0.1400	0.3875	0.5150	0.0500	0.2500
Hit@3	0.3700	0.5625	0.8450	0.1600	0.4900
Hit@5	0.5000	0.6500	0.9500	0.2550	0.5450
Precision@10	0.0985	0.2313	0.1718	0.0410	0.1208
F1@10	0.1616	0.3208	0.2830	0.0694	0.1888
MRR	0.2594	0.4730	0.6317	0.1302	0.3697
Fully Covered	0.5400	0.5500	0.9750	0.2150	0.5800
Partial+	0.7050	0.8500	0.9850	0.3800	0.7050
Not Covered	0.2950	0.1500	0.0150	0.6200	0.2950
Overlap	0.6562	0.7968	0.9931	0.4205	0.6897

Table 5. Search with Context Budget Limit with TPE | Top-5

Rank	Trial	Score	Chunk Size	Similarity k	BM25 k	Final k	Estimated Context Chars
1	72	0.5211	2000	10	41	11	22,000
2	61	0.5190	2000	9	52	11	22,000
3	81	0.5188	2000	10	46	11	22,000
4	82	0.5188	2000	10	45	11	22,000
5	86	0.5188	2000	10	48	11	22,000

The optimal results were achieved using optimal settings and tested with non-overlapping datasets from the config tuning process.

Table 6. Search with Context Budget Limit with Random Search | Top-5

Rank	Trial	Score	Chunk Size	Similarity k	BM25 k	Final k	Estimated Context Chars
1	73	0.5383	2000	15	73	11	22,000
2	6	0.5168	2000	9	43	11	22,000
3	29	0.5026	2000	12	91	10	20,000
4	95	0.4996	2000	4	57	11	22,000
5	2	0.4994	2000	4	90	11	22,000

The following result uses

```

chunk_size = 2000 (chars, around 500 token )
overlap = 400 (chars, around 100 token )
similarity_k = 15
bm25_k = 73
final_k = 11
rrf_k = 60

```

Table 7. Optimal Result with our Tuning Approach | Budget Limit + Random

Metrics	CUAD	PrivacyQA	ContractNLI	MAUD	Mixed Validation
Recall@10	0.6193	0.6808	0.9767	0.3097	0.6286
Hit@1	0.1400	0.3875	0.5150	0.0450	0.2400
Hit@3	0.3700	0.5625	0.8450	0.1600	0.5000
Hit@5	0.4700	0.6625	0.9500	0.2700	0.5900
Precision@10	0.0960	0.2275	0.1713	0.0445	0.1188
F1@10	0.1573	0.3162	0.2821	0.0753	0.1855
MRR	0.2558	0.4713	0.6317	0.1310	0.3751
Fully Covered	0.5500	0.5625	0.9750	0.2350	0.5750
Partial+	0.7300	0.8375	0.9850	0.4300	0.7250
Not Covered	0.2700	0.1625	0.0150	0.5700	0.2750
Overlap	0.6759	0.8065	0.9931	0.4595	0.6896

Note that the above result is based on sampled dataset without overlap with config dataset. Detail steps are included in replication.pdf.

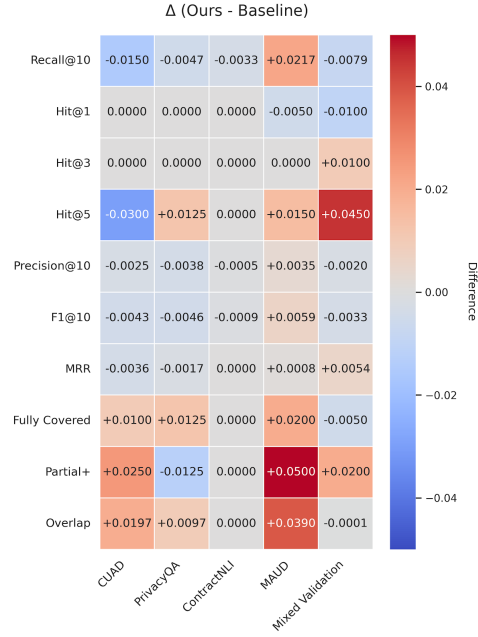


Figure 1. Heatmap of Comparison between Our Searching Approach and the Baseline Approach

6. Reflection and Limitation

6.1 Optimisation Stability and Search Method Sensitivity

In our preliminary experiments, configurations obtained via Random Search achieved slightly higher scores than those found by TPE. However, this observation should not be interpreted as evidence of the general superiority of Random Search. Rather, it reflects the behaviour of the optimisation methods under the specific conditions of this study.

One contributing factor is the structure of the search space, which includes several discrete hyperparameters, such as chunk size, similarity top-k, BM25 top-k, and final top-k. Additionally, each trial was evaluated on a relatively small subset of queries, which may introduce noise into the objective function and reduce its stability. Under such conditions, model-based methods like TPE may struggle to accurately estimate promising regions of the search space, especially in early stages with limited observations. This can lead to premature exploitation of suboptimal regions.

In comparison with TPE, Random Search maintains border exploration and may therefore have a higher chance of encountering well-performing configurations within a constrained budget.

A more conclusive comparison between Random Search and TPE would require repeated runs with different random seeds, a larger evaluation subset, and statistical comparison across runs. Therefore, the current result should be viewed as an empirical observation rather than a general conclusion about optimisation methods.

The effect of chunk size should also be interpreted with caution. Larger chunks may improve recall and coverage because they contain more surrounding text, but this does not necessarily mean that the retrieval ranking itself is more accurate. Conversely, smaller chunks may provide more precise evidence spans but may require retrieving more chunks to achieve comparable coverage. Since the current study does not isolate chunk size through controlled experiments, the relationship between chunk size and retrieval quality remains inconclusive.

6.2 Retrieval Efficiency and BM25 Depth

While increasing the BM25 retrieval depth effectively improved evidence coverage with limited latency impact, the same approach for dense similarity retrieval provided less consistent benefits and appeared less resource-efficient.

This result suggests that lexical matching remains highly valuable for legal document retrieval. Legal queries and relevant evidence often share exact terminology, clause names, or domain-specific expressions, making BM25 particularly useful for recovering candidate evidence passages. Therefore, allocating more of the retrieval budget to BM25 can be a cost-effective strategy compared with simply increasing dense retrieval depth. However, this finding is specific to the datasets and retrieval pipeline used in this study. In particular, our experiments were conducted using a 0.6B embedding model, which may have constrained the effectiveness of dense retrieval compared to larger, more robust architectures.

Different legal corpora, embedding models, or query types may require a different balance between sparse and dense retrieval. Future work should evaluate whether the same BM25-heavy configuration remains effective across broader legal retrieval tasks.

7. Conclusion

This report examined configuration tuning for the retrieval component of a lightweight local legal RAG system under a fixed context budget. The results show that unconstrained tuning tends to favour larger retrieved contexts, which can improve coverage but may exceed practical inference limits.

Under the budget-constrained setting, increasing BM25 top-k was a cost-effective way to improve evidence coverage, while increasing dense similarity top-k showed less consistent benefit. This suggests that exact lexical matching remains important for legal RAG, especially under local resource constraints.

Future work should evaluate these findings on larger validation sets and additional legal corpora. Further experiments could also isolate the effects of chunk size and overlap, and explore adaptive retrieval strategies that adjust sparse and dense retrieval depth based on query type.

Future work should extend this study to the full end-to-end RAG pipeline, including the generation component, to evaluate how retrieval improvements translate to answer quality. It would also be useful to evaluate these findings on larger validation sets and additional legal corpora. Further experiments could isolate the effects of chunk size and overlap, and explore adaptive retrieval strategies that adjust sparse and dense retrieval depth based on query type.

Reference

- [1] D. Kim, B. Kim, D. Han, and M. Eibich, “AutoRAG: Automated framework for optimization of retrieval-augmented generation pipeline,” *arXiv preprint* arXiv:2410.20878, 2024.
- [2] J. Fu *et al.*, “AutoRAG-HP: Automatic online hyper-parameter tuning for retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2024.
- [3] S. Es *et al.*, “RAGAS: Automated evaluation of retrieval-augmented generation,” in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, 2024.
- [4] J. Saad-Falcon *et al.*, “ARES: An automated evaluation framework for retrieval-augmented generation systems,” in *Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [5] B. Smith, “Evaluating chunking strategies for retrieval,” *Chroma Research Technical Report*, 2024.
- [6] C. Merola *et al.*, “Evaluating advanced chunking strategies for retrieval-augmented generation,” *arXiv preprint* arXiv:2504.19754, 2025.
- [7] N. Pipitone and G. H. Alami, “LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain,” *arXiv preprint* arXiv:2408.10343, 2024.

Appendix

Repository: https://github.com/MironChen/PowerLLM-config_tuning